근거를 찾고, 흐름을 연결하고, 결과를 확인하기

Legal Intake · 기본 설계 이해를 위한 공부 자료

한 번에 이해할 핵심

LangChain + LangGraph + LangSmith + E5 임베딩 + memvid를 사용해, 검색한 근거로 Llama가 답하는 작은 서비스를 설계합니다.

첫 목표는 작게

합성 상담 준비 안내문 10개, 평가 질문 20개로 시작합니다. 기존 안내 API 하나에 먼저 연결하고, 앱 API는 최종 최대 11개로 유지합니다.

현재 상태

Claude 검수 전 설계 초안입니다. 새 검색 기능은 아직 구현되지 않았습니다. 기존 API와 L01/L02 구현 기록을 새 RAG의 검증 결과로 혼동하지 않습니다.

2026.11.06 | 4:3 태블릿 가로형 | 다크 모드 | 검수 후 최종 개발안 확정

이 PDF를 읽는 순서

용어를 외우기보다 한 질문이 지나가는 길을 따라 읽으세요.

01 · 큰 그림

3~8쪽
일곱 개념의 역할과 LangChain·LangGraph·LangSmith의 차이를 먼저 봅니다.

02 · 질문 한 건의 여행

9~17쪽
문서 적재, E5 임베딩, memvid 검색, 근거 분기, Llama 생성의 연결을 봅니다.

03 · 실무에서 지킬 경계

18~24쪽
문맥·보안·비용·취소·추적·평가·API 범위를 확인합니다.

04 · 실제 개발 준비

25~30쪽
모듈 구조, 개발 순서, Claude 검수, 복습과 공식 근거를 확인합니다.

HTML의 검색 비교 버튼은 PDF에서 동작하지 않습니다. 대신 세 가지 고정 사례를 설명합니다.

일곱 가지는 서로 다른 층에 있습니다

모든 이름을 별도 서버로 만들 필요는 없습니다.

개념	맡는 일	우리 설계의 최소 사용
LangChain	프롬프트·모델·검색 인터페이스 조합	한 node 안에서 구조화된 초안 만들기
LangGraph	순서·분기·요청별 state 관리	검색 뒤 근거 유무에 따라 두 갈래로 이동
LangSmith	실행 추적과 품질 평가	검색 방식·시간·오류·버전을 비교
RAG	검색 근거를 넣어 답하는 전체 방식	우리 안내문을 근거로 답변
임베딩	텍스트 의미를 숫자 벡터로 표현	문서와 질문을 같은 모델로 encode
벡터 RAG	벡터 검색을 이용하는 RAG	표현이 다른 질문에서도 관련 자료 찾기
memvid	문서·벡터·검색 인덱스 저장	마스킹된 자료를 .mv2에 보관

RAG는 방식, memvid는 저장·검색 부품입니다. memvid를 이용해 RAG를 구성합니다.

LangChain · LangGraph · LangSmith

작업을 만든다 / 순서를 연결한다 / 결과를 살펴본다.

LangChain = 작업 부품

질문과 근거를 템플릿에 넣고 모델을 호출합니다. 모델 결과를 Pydantic 구조로 검증하는 연결도 만듭니다.

LangGraph = 진행표

retrieve 다음에 generate를 할지, 추가 자료 요청으로 끝낼지 정합니다. LLM이 실행 순서를 마음대로 고르지 않습니다.

LangSmith = 실행 기록·평가

어떤 문서를 찾았는지, 어느 단계가 느렸는지, 프롬프트를 바꾼 뒤 좋아졌는지 비교합니다.

결합 방식

LangGraph node 안에서 LangChain을 사용합니다. LangSmith는 서비스와 graph 실행을 관찰합니다. 보안 검사는 서버가 강제합니다.

LangGraph가 LangChain을 반드시 요구하는 것은 아닙니다. 이번 설계에서는 함께 사용합니다.

LangChain에서 먼저 공부할 것

메시지·프롬프트·LCEL·구조화 출력만 작게 사용합니다.

메시지
역할을 구분

→

템플릿
질문·근거 주입

→

모델
Llama 생성

→

스키마
형식 검증

메시지의 역할

시스템 지시는 서버가 정하고, 질문과 검색 자료는 데이터로 전달합니다. 후기나 안내문 안의 명령문을 시스템 지시로 승격하지 않습니다.

LCEL은 짧은 연결에 사용

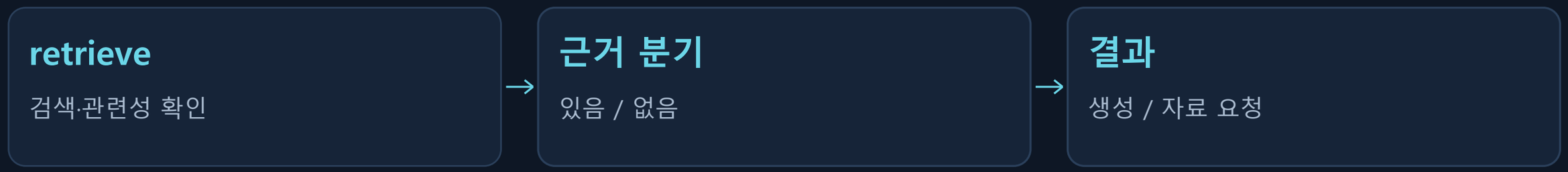
prompt → model → schema라는 한 작업의 연결에 사용합니다. 전체 업무의 근거 분기와 node 실행 순서는 LangGraph가 담당합니다.

무엇을 검증하나

답변·준비자료·확인할 질문·근거 ID를 구조로 받습니다. JSON 형식이 맞는 것과 사실이 맞는 것은 별개입니다.

LangGraph는 분기 하나로 시작합니다

첫 버전은 고정 workflow이며 자율 에이전트가 아닙니다.



근거가 있는 길

retrieve → generate → END. 관련 자료가 있다고 서버가 판정하면 LangChain chain을 사용해 Llama를 최대 한 번 호출합니다.

근거가 없는 길

retrieve → ask_for_materials → END. 추가 자료 요청 템플릿으로 끝내고 생성 모델은 호출하지 않습니다.

첫 버전에서 단순하게 유지

반복 재검색, 자동 재시도, 다중 에이전트, 영속 대화 checkpoint를 넣지 않습니다. 요청마다 별도 state를 만듭니다.

검색 장애는 "근거 없음"과 다릅니다. 기술 오류로 구분해 원인을 숨기지 않습니다.

LangSmith에서 무엇을 볼까요?

검색기가 아니라 추적·평가 도구입니다.

Trace: 한 요청의 기록

질문 임베딩 → 검색 → 생성 → 검증 단계의 시간과 결과를 확인합니다. 요청 trace ID로 한 흐름을 묶습니다.

Dataset: 고정 질문 묶음

한국어 합성 질문과 기대 문서 ID를 준비합니다. 같은 질문으로 키워드 검색과 벡터 검색을 비교합니다.

Experiment: 버전 비교

청크 길이·임베딩 모델·프롬프트·인덱스 버전을 바꾼 결과를 비교합니다. 좋아졌다는 판단에 근거를 남깁니다.

실제 연동 완료의 기준

SDK 설치나 mock 테스트만으로 완료라 하지 않습니다. 원격 trace와 합성 dataset 실험이 보이는지 따로 확인합니다.

키가 없는 환경의 local 시험과 실제 LangSmith 원격 연동 성공을 구분합니다.

RAG와 벡터 RAG를 구분하기

검색한 자료를 생성 모델이 읽도록 전달하는 것이 핵심입니다.

RAG의 두 부분

Retrieve는 질문과 관련된 자료를 찾는 단계, Generate는 그 자료를 근거로 답변을 만드는 단계입니다. 모델을 재학습시키는 것과 다릅니다.

벡터 RAG

문서와 질문을 임베딩하고 벡터 유사도로 근거 후보를 찾습니다. 검색 결과의 텍스트를 Llama에 전달하며 벡터 자체를 답변 모델이 읽는 것은 아닙니다.

키워드 검색도 기준선

정확한 용어와 문서 번호에는 키워드 검색이 유용할 수 있습니다. 벡터 검색이 언제나 이기는 것으로 가정하지 않고 같은 질문으로 비교합니다.

memvid의 위치

memvid가 문서와 검색 인덱스를 맡습니다. 첫 범위에 다른 벡터 DB를 동시에 추가하지 않아 구성 요소를 줄입니다.

하나의 예제로 끝까지 따라가기

아래는 합성 학습 예시이며 실제 법률 판단이 아닙니다.

사용자 질문

"집주인이 돈을 안 돌려줘요. 상담 전에 무엇을 준비하죠?"
일상적인 표현으로 질문합니다.

찾고 싶은 자료

GUIDE-001 / 임대차 보증금 반환 상담 준비 / v1
자료에 계약서·입금 기록 준비 항목이 있습니다.

만들 답변

"상담 전에 계약서와 입금 자료를 정리하세요."
준비자료와 확인할 질문, 실제 사용한 출처 ID를 함께 만듭니다.

검사할 것

검색한 자료에 없는 승소 보장이나 기한 판단을 만들지 않았는가? 출처 ID가 실제 검색 결과 안에 있는가?

이 예시는 처리 방식을 이해하기 위한 것이며 검색 성능 점수를 측정한 결과가 아닙니다.

미리 해 둘 일: 문서 적재

준비한 자료를 검색할 수 있는 상태로 만드는 흐름입니다.

문서 읽기
합성 안내문 10개
→
검사·마스킹
저장 전 처리
→
분할
문단·토큰 기준
→
임베딩·저장
E5 → memvid

자료 범위를 먼저 고정

처음에는 합성 상담 준비 안내문만 넣습니다. 실제 접수 원문·개인 사건 기록·서비스 후기를 자동으로 검색 지식에 섞지 않습니다.

문서에 붙일 꼬리표

documentId, chunkId, sourceVersion, collectionId를 보존합니다. 나중에 어떤 문서의 어느 조각이 답변 근거였는지 찾아야 합니다.

적재를 질문마다 반복하지 않기

문서 내용과 모델 설정이 같으면 만든 벡터를 재사용합니다. 재적재·버전 교체와 실패 복구는 별도 적재 script가 담당합니다.

문서를 왜 잘게 나눌까요?

검색 단위는 문서 전체가 아니라 의미가 유지된 작은 청크입니다.

너무 길면

E5-large-instruct는 최대 512토큰으로 입력을 제한합니다. 긴 문서를 통째로 넣으면 뒤가 잘려 필요한 내용을 놓칠 수 있습니다.

너무 짧으면

"자료를 준비하세요" 같은 문장만 남으면 무엇을 위한 자료인지 맥락이 사라집니다. 제목·절·문단 구조를 보존해야 합니다.

기본 실험값

E5 tokenizer 기준 300토큰 청크와 40토큰 겹침으로 시작합니다. 최적값이라는 뜻은 아닙니다. instruction을 포함한 최종 길이를 실제 tokenizer로 확인합니다.

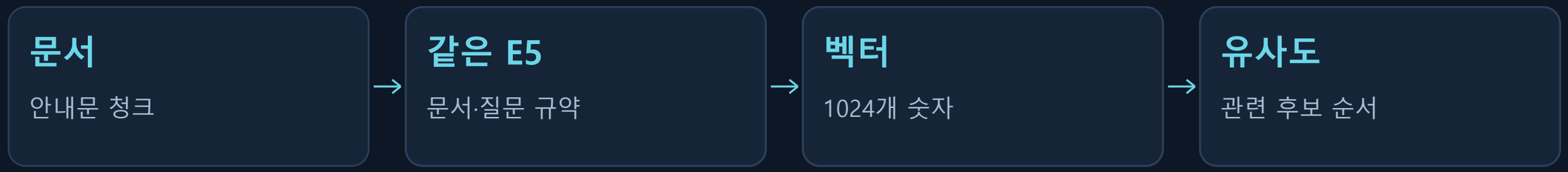
겹침과 버전

이웃 청크가 약간 겹치면 문맥 단절을 줄일 수 있지만 중복 검색도 늘 수 있습니다. chunkerVersion과 출처를 기록하고 평가로 조정합니다.

> 한국어 글자 수와 모델 토큰 수는 다릅니다. 300글자라고 300토큰으로 세지 않습니다.

임베딩을 숫자로 이해하기

텍스트의 의미를 검색 가능한 벡터로 바꿉니다.



문서와 질문은 같은 공간

문서에 쓴 임베딩 모델과 질문에 쓴 모델은 같아야 합니다. 모델 revision·차원·정규화 규약도 함께 고정합니다.

E5의 질문과 문서 입력은 다름

질문에는 "어떤 자료를 찾을 것인지"를 설명하는 검색 instruction을 넣습니다. 문서에는 질문용 instruction을 그대로 붙이지 않습니다.

코사인 유사도는 방향의 비슷함

같은 모델 공간에서 두 벡터의 방향을 비교해 관련 자료 후보를 정렬합니다. 점수는 정답 확률이 아닙니다. 이번에는 최소 개념만 공부하며 개인 검색식이나 별도 리서치 설계는 추가하지 않습니다.

E5 검색과 Llama 생성은 함께 쓸 수 있습니다

두 모델은 같은 회사나 같은 계열일 필요가 없습니다.

구분	E5-large-instruct	Llama 계열
역할	검색용 벡터 생성	검색한 텍스트를 읽고 답변 생성
입력	마스킹한 문서 청크 또는 질문	서버 지시 + 질문 + 검증된 근거
출력	1024차원의 숫자 벡터	안내 답변·준비자료·질문 초안
이번 선택	로컬 실행 우선 검토	정확한 모델 ID·정밀도는 미확정
검증	한국어 검색 품질·길이·차원	한국어 답변·JSON 형식·근거 충실도

E5의 출처

Microsoft 연구진이 개발한 다국어 임베딩 계열입니다. intfloat는 배포 계정명입니다. 로컬 실행은 별도로 준비하고 검증해야 합니다.

> 현재 코드의 실제 생성 어댑터는 OpenAI SDK입니다. Llama 연결은 아직 개발 예정입니다.

임베딩 후보를 같은 기준으로 보기

기본안은 E5 우선 검토이며 Qwen은 실행 범위에서 제외합니다.

항목	E5-large-instruct	Qwen3-Embedding-8B
개발 계열	Microsoft 연구진 / E5	Qwen 팀 / Alibaba 계열
규모·차원	약 0.6B / 1024차원	8B / 기본 최대 4096차원
입력 길이	최대 512토큰	32K토큰
라이선스 표기	MIT	Apache-2.0
우리 선택	우선 구현·평가 대상	대표님 방침에 따라 실행 제외

선택의 의미

E5가 이 프로젝트에서 더 낫다는 실측 결론은 아직 없습니다. 실행 부담과 한국어 검색 품질을 확인한 뒤 최종안을 고정합니다.

보안의 기준

로컬 추론과 외부 API 호출은 데이터 이동 경로가 다릅니다. 개발사 이름만으로 유출 여부를 단정하지 않습니다.

OpenAI small / large 표를 해석하는 법

사양과 특정 실험 결과를 구분합니다.

모델	기본 차원	제공 방식
text-embedding-3-small	1536	OpenAI API
text-embedding-3-large	3072	OpenAI API

14/20·18/20·19/20·20/20은?

전달된 표에는 평가 질문·문서·지표 이름·top-k 조건이 없어 확인 불가입니다. 우리 프로젝트의 측정 결과나 모든 한국어 검색의 성능으로 인용하지 않습니다.

Llama와 함께 사용할 수 있나요?

가능합니다. 다만 임베딩할 텍스트가 API 제공자에게 전송되므로 로컬 E5 구성과 구분해야 합니다. 현재는 대안 설명이며 자동 채택하지 않습니다.

> 기본 차원과 차원 축소 기능은 OpenAI 공식 임베딩 문서에서 확인했습니다.

memvid는 무엇을 담고 있나요?

문서 텍스트·출처·벡터·검색 인덱스를 함께 관리하는 부품입니다.

분류	최소 저장 정보	필요한 이유
본문	마스킹한 safeText	생성 모델이 읽을 근거
출처	documentId / chunkId / sourceVersion	근거와 원본 문서를 연결
자료 범위	collectionId / documentType	후기와 공식 안내 자료의 혼합 방지
모델 규약	model / revision / dimension	다른 의미 공간 혼합 방지
인덱스	indexVersion / chunkerVersion	재적재·실험 비교·복구

Windows 연결 방식

memvid 내장 로컬 임베딩 제한과 별개로, E5 provider가 만든 외부 벡터를 전달하는 어댑터를 검토합니다. 실제 SDK의 적재·질의 지원은 먼저 통합 시험합니다.

> memvid의 직접 생성 기능으로 보안 검사를 우회하지 않고 검색 결과를 우리 서비스에 반환합니다.

검색 결과가 있으면 무조건 답할까요?

top-3 결과와 "쓸 수 있는 근거"는 다릅니다.

일상 표현 질문

"집주인이 돈을 안 돌려줘요."
의미 검색이 관련 안내문을 찾는지 확인합니다. 키워드 검색과 같은 자료로 비교합니다.

정확한 용어 질문

"보증금 반환 상담 준비."
키워드도 충분히 잘 찾을 수 있습니다. 벡터 검색의 우위를 미리 가정하지 않습니다.

자료에 없는 질문

후보가 반환돼도 관련성이 부족하면 무근거로 판단합니다. 추가 자료를 요청하고 Llama를 호출하지 않습니다.

관련성 기준

임의의 cosine 0.8을 공통 기준으로 쓰지 않습니다. 모델별 점수 분포와 합성 평가 질문을 보고 기준을 보정합니다.

이 페이지의 세 사례는 설명용입니다. 실제 검색 점수나 정답률을 표시한 결과가 아닙니다.

요청별 state와 문맥은 어떻게 지키나요?

같은 세션의 동시 요청도 서로 다른 실행입니다.

RequestContext

서버가 만든 trace ID, 세션 그룹 키, 분석 대상 snapshot, profile, 정책 버전을 묶습니다. 요청마다 독립적인 값을 만듭니다.

GraphState

safeQuestion, evidence, evidenceStatus, draft를 요청마다 별도로 전달합니다. 공유 client에 current_message 같은 현재 질문 속성을 두지 않습니다.

Envelope와 반환 직전 검사

생성 단계가 보존한 문맥과 본문을 반환 직전에 검증합니다. 다른 요청의 결과를 현재 문맥으로 새로 묶어서 검사하는 실수를 피합니다.

세션의 한계

세션 ID는 누적 위험의 그룹 키이며 로그인·소유권 증명이 아닙니다. 초기 graph에는 영속 대화 checkpoint를 붙이지 않습니다.

> 문맥 검사는 상태 격리를 보조합니다. 모델이 잘못 답한 사실 자체를 자동 판별하는 만능 검사는 아닙니다.

구조화 출력과 출처 검증

형식 검증 뒤에도 근거와 정책을 확인해야 합니다.

내부 필드	무엇을 담나	검증할 것
answer	사용자에게 전달할 안내	길이·정책·최종 마스킹
preparationItems	준비자료 목록	자료에 없는 사실을 만들지 않음
questions	상담 전 확인할 질문	민감정보 공개 유도 여부
citationIds	사용한 청크의 ID 목록	실제 검색 결과 ID의 부분집합
evidenceStatus	FOUND / NOT_FOUND	무근거 분기는 서버가 확정

기존 API 계약은 먼저 유지

위 구조는 내부 DTO와 평가에 먼저 사용합니다. 기존 answer/model/intakeId 응답은 유지하고 외부 필드 추가는 별도 계약 변경으로 검토합니다.

> Llama의 native schema 지원을 확인합니다. JSON 파싱·검증 실패 출력을 그대로 공개하지 않습니다.

보안 action 9개를 어디에 두나요?

graph와 chain이 생겨도 필수 검사를 선택 사항으로 만들지 않습니다.

경계	action	역할
진입·반환	validate_request_context	요청·snapshot·응답의 연결 확인
입력 전	check_session_risk	기존 세션 차단 확인
질문·근거	check_input_policy	데이터 속 공격 지시 검사
저장·전송·반환	mask_personal_data	정의된 개인정보 패턴 치환
호출 진입	check_generation_admission	동시 실행·유한 대기 제어
유료 단계	check_request_budget	원자 예약·정산
전체 초안	check_output_policy	출력 정책과 오류 처리
향후 M01/M02	check_marketing_response	보장·과장 표현 검토
확정 정책 차단	record_risk_event	중복 없는 위험 이벤트

L01/L02 외 action과 실행 pipeline은 후속 작업입니다. 9개 모두 완료했다고 표시하지 않습니다.

비용·동시성·취소는 누가 책임지나요?

모델 node 바깥의 실행 서비스가 호출의 전체 수명을 관리합니다.

용량 확보 실행 자리 → **예산 예약** 원자 처리 → **모델 호출** await / worker → **정산·반환** finally 정리

로컬과 원격을 구분

로컬 E5/Llama는 API 토큰 요금이 없어도 메모리·계산 자원·동시성 제한이 필요합니다. 원격 임베딩은 생성과 별도의 유료 단계로 취급합니다.

이벤트 루프를 막지 않기

동기 GPU/CPU inference를 async def 안에서 직접 오래 실행하지 않습니다. 비동기 client 또는 제한된 worker 실행 경계를 사용합니다.

취소가 곧 연산 종료는 아님

Python task를 취소해도 native inference가 계속될 수 있습니다. 실제 호출 종료 여부를 확인하지 않고 슬롯을 성급히 반환하지 않도록 검증합니다.

로컬 모델이어도 trace는 별도입니다

입력·출력·metadata·예외의 전송 경로를 모두 확인합니다.

기록할 정보

trace ID, node 이름, 문서 ID, 검색 모드, 시간, 판정 코드, 모델·프롬프트·인덱스 버전 중심으로 시작합니다.

기본 숨김

실제 사건 원문, 연락처, 키, 세션 원문, 원문 예외를 보내지 않습니다. 입력·출력을 숨겨도 metadata와 오류를 따로 점검해야 합니다.

합성 데이터로 학습

공부용 질문·문서에 한해 내용을 기록하는 실험을 분리합니다. 실제 의뢰인 자료를 평가 dataset에 자동 추가하지 않습니다.

기록 장애와 보안 장애

LangSmith 기록 실패를 이유로 보안 검사를 우회하지 않습니다. 기록 실패 로그에 원문을 다시 쓰지도 않습니다.

로컬 추론과 원격 LangSmith 전송은 다른 결정입니다. 모델을 로컬로 돌린다고 trace도 로컬인 것은 아닙니다.

검색이 좋아졌는지 어떻게 알까요?

고정된 한국어 질문과 기대 문서로 비교합니다.

평가 항목	첫 확인 방법	주의할 점
Hit@3	20개 질문 중 기대 문서가 top-3에 포함된 비율	질문별 정답 문서 ID를 미리 정함
무근거 분기	자료 없는 질문에서 생성 0회·출처 비어 있음	항상 top-k를 반환하는 것과 구분
출처	모든 citation ID가 검색 결과 안에 있음	ID 존재만으로 문장 사실성이 보장되지는 않음
지연·자원	임베딩·검색·생성 시간을 분리	모델 로딩·배치·장치를 함께 기록
안전성	합성 PII가 저장·응답·trace에 없는지	오류 경로·취소 경로도 확인

기본 목표와 실제 결과

Hit@3 80%는 초기 목표 예시이며 달성 결과가 아닙니다. 평가용 20개와 별도 확인용 10개를 두고 같은 질문으로 버전을 비교합니다.

앱 API는 최대 11개로 유지합니다

검색·적재·평가는 먼저 내부 모듈과 script로 만듭니다.

기능 묶음	경로 요약	목표 개수
현재 접수 MVP	등록 / 조회 / 접수별 guidance	3
상태 확인	GET /health	1
게시글	POST·GET /api/posts, GET /api/posts/{postId}	3
후기	POST·GET /api/posts/{postId}/reviews	2
AI 마케팅	게시글 요약 / 후기 답글 초안	2
최종 합계	메서드와 경로 조합으로 계산	11

첫 연결은 guidance 하나

현재 실행되는 API는 기존 3개입니다. 나머지는 목표 계약입니다. 게시판·M01/M02는 같은 내부 부품을 재사용하고 마케팅 검사 재검증 뒤 활성화합니다.

> 이전 상세 설계의 21개는 후보 목록입니다. 단독 검색 API를 더해 11개를 넘기지 않습니다.

개발 파일을 역할로 나누기

이 배치는 예정 구조이며 파일이 이미 있다는 뜻은 아닙니다.

영역	예정 파일	담당 책임
retrieval	ingest.py / embeddings.py	자료 준비·E5 문서/질문 encode
retrieval	memvid_store.py / retriever.py	벡터 저장·검색·Document 반환
chains	guidance_chain.py	프롬프트·Llama·구조화 출력
graphs	guidance_graph.py	retrieve / generate / 자료 요청 분기
observability	tracing.py	기록 전 처리·trace 연결
service	guidance_service.py	기존 API 계약·필수 검사·호출 정리
scripts	ingest_guides.py / evaluate_retrieval.py	적재와 반복 가능한 평가

초기에는 같은 책임의 어댑터를 여러 겹 만들지 않습니다. Claude가 과도한 추상화도 검수합니다.

기본 개발 순서: B01부터 B06까지

필수 보안 경계를 선행한 뒤 작은 묶음으로 진행합니다.

단계	만들 것	끝났다는 증거
선행	입력·마스킹·출력·비용·자원 연결	차단·오류·취소에서 우회·누출·누수 없음
B01	합성 문서·분할·출처 metadata	token 상한·재적재 중복 검증
B02	E5 + memvid 적재·검색	실제 벡터 encode와 .mv2 round-trip
B03	LangChain + Llama + 내부 DTO	실제 템플릿·형식·출처 검사
B04	작은 LangGraph 실행	정상 생성 1회 / 무근거 0회 / A/B 격리
B05	LangSmith trace·평가	원격 trace와 dataset 실험 확인
B06	guidance 연결·후속 재사용	기존 API 회귀·M 기능 활성 전 검사

fake 통과와 실제 라이브러리 통합 성공, 실제 모델 실행, 원격 연동 성공을 따로 기록합니다.

Claude 검수 후 최종안을 확정합니다

지금은 검토용 초안이며 새 검색 구현을 시작하는 단계가 아닙니다.

설계 전달
구조·공부 문서
→
Claude 검수
읽기 전용
→
지적 반영
수용·보류·반박
→
최종 확정
그다음 구현

Claude가 볼 핵심

역할 중복, E5 입력 규약, memvid 외부 벡터 호환성, 근거 분기, trace 노출, Windows 실행 자원, 취소 정리와 API 제한을 확인합니다.

아직 결정할 사항

Llama의 정확한 모델 ID·정밀도·서버, E5 고정 revision, memvid SDK 조합, 한국어 관련성 기준, LangSmith project·전송 규칙을 확정해야 합니다.

구현 착수 기준

P0/P1 지적을 해소하거나 이유 있는 보류로 정리합니다. 최종 개발안 v1에 범위·환경·테스트를 고정하고 대표님 확인 뒤 구현합니다.

오해하기 쉬운 질문 여섯 가지

다른 사람에게 설명할 수 있으면 구조를 이해한 것입니다.

질문	답변
memvid가 있으면 RAG가 필요 없나요?	memvid로 검색한 근거를 넣어 답하면 그것이 RAG입니다.
Llama를 쓰면 임베딩도 Llama여야 하나요?	아닙니다. E5 검색 결과의 텍스트를 Llama가 읽을 수 있습니다.
1024차원끼리는 서로 바꿔 써도 되나요?	안 됩니다. 모델 공간·revision·전처리가 다를 수 있습니다.
벡터 검색은 언제나 키워드보다 좋나요?	아닙니다. 같은 질문·문서로 비교해야 합니다.
JSON이 맞으면 답변이 정확한가요?	형식·근거·정책은 서로 다른 검사입니다.
로컬 모델이면 trace도 로컬인가요?	아닙니다. LangSmith의 전송 설정은 별도로 확인합니다.

자료가 없을 때 답하지 않는 흐름도 기능의 일부입니다. 정답처럼 꾸민 답변보다 원인을 드러냅니다.

전체 연결을 한 화면에 그리면

위에서 자료를 준비하고, 아래에서 질문에 답합니다. 모든 칸은 검수할 설계입니다.

사전 적재 | 자료가 바뀔 때 준비

문서 준비
검사·마스킹·분할

→

E5 임베딩
문서 벡터 생성

→

memvid
텍스트·출처·벡터 저장

질문 응답 | LangGraph가 고정 순서와 근거 분기를 진행

입력 검사
문맥·세션·마스킹

→

retrieve
E5 질문 → memvid

→

generate
LangChain + Llama

→

출력 검사
출처·정책·문맥

근거가 없으면 generate로 가지 않고 추가 자료 요청으로 끝냅니다. 생성 호출은 0회입니다.

LangSmith | 위 단계들의 실행을 관찰하고 비교

질문 임베딩·검색·생성의 시간과 결과를 기록합니다. 기록 전 개인정보를 처리하고, 같은 합성 dataset으로 버전을 평가합니다.

공식 근거와 다시 볼 문서

링크는 PDF에서 누르면 열립니다. 외부 링크는 읽을 때만 연결됩니다.

LangGraph 역할과 구성 https://docs.langchain.com/oss/python/langgraph/overview

고정 workflow와 agent https://docs.langchain.com/oss/python/langgraph/workflows-agents

LangChain 메시지 https://docs.langchain.com/oss/python/langchain/messages

LangChain 구조화 출력 https://docs.langchain.com/oss/python/langchain/structured-output

LangChain 텍스트 분할 https://docs.langchain.com/oss/python/integrations/splitters/index

LangSmith tracing / evaluation https://docs.langchain.com/langsmith/evaluation

LangSmith 민감정보 기록 제어 https://docs.langchain.com/langsmith/mask-inputs-outputs

memvid SDK 2.0.160 https://pypi.org/project/memvid-sdk/2.0.160/

E5 공식 모델 카드 https://huggingface.co/intfloat/multilingual-e5-large-instruct

Microsoft 연구진 E5 보고서 https://arxiv.org/html/2402.05672v1

Qwen 비교용 모델 카드 https://huggingface.co/Qwen/Qwen3-Embedding-8B

OpenAI 임베딩 공식 문서 https://developers.openai.com/api/docs/guides/embeddings

이 문서는 기본 설계 설명입니다. 실제 한국어 검색 우열·하드웨어 실행·새 RAG 통과 결과는 아직 없습니다.